Top 5 Enterprise DBA Tools

You Can Build Free with Claude Code

Replace $15,000+ /year in Enterprise Licenses

Multi-Database Support: PostgreSQL • MongoDB • MySQL / MariaDB

Built With: Python • Flask / FastAPI • React • Chart.js • SQLite Metadata Store

Target: DBA Teams on AlmaLinux 8/9 — Private Cloud — /apps Filesystem

Contents

Why Build Your Own DBA Tools?

Enterprise database administration tools carry significant licensing costs — often $2,000 to $10,000+ per server per year. For a DBA managing 10–30 database instances across PostgreSQL, MongoDB, and MySQL/MariaDB on private cloud VMs, these costs compound rapidly. Many of these tools provide features that a skilled DBA only uses 20–30% of, yet the entire license must be purchased.

Claude Code changes this equation entirely. As an agentic AI coding assistant that runs in your terminal, Claude Code can scaffold, implement, and iterate on full-stack applications by understanding your requirements in natural language. You describe what you need, Claude Code writes the code, and you review, refine, and deploy — all without writing boilerplate from scratch.

What Makes These 5 Apps Ideal for Claude Code?

- They are internal tools — no need for commercial polish, just functional reliability
- They follow well-known patterns — dashboards, CRUD, scheduled jobs, REST APIs
- They use standard tech stacks — Python, Flask/FastAPI, React, Chart.js, SQLite
- They need database connectivity — Claude Code excels at writing DB adapter code
- They are self-contained — deployable on your existing AlmaLinux VMs under /apps

Enterprise Tools Cost Comparison

Enterprise Tool	Category	Annual Cost (Approx.)	Your DIY Cost
Datadog DB Monitoring	Monitoring Dashboard	$840–$2,760 / host / yr	$0 (self-hosted)
SolarWinds DPA	Query Performance	$2,000–$6,000 / instance / yr	$0 (self-hosted)
New Relic DB	Monitoring	$600–$1,800 / host / yr	$0 (self-hosted)
EverSQL / Aiven	Query Tuning Advisor	$200–$2,400 / yr	$0 (self-hosted)
Commvault / Veeam	Backup Management	$3,000–$15,000 / yr	$0 (self-hosted)
Idera DB PowerSuite	Health Audit	$2,000–$5,000 / instance / yr	$0 (self-hosted)
Redgate SQL Doc	Documentation	$500–$1,500 / user / yr	$0 (self-hosted)
ScaleGrid / Percona PMM Enterprise	Combined Platform	$1,200–$6,000 / yr	$0 (self-hosted)

COST SAVINGS

For a DBA supporting 10 database instances, enterprise tool licenses typically total $12,000 — $50,000+ per year. Every app in this guide can be built and deployed for $0 in infrastructure cost (runs on your existing VMs) and a few hours of Claude Code time.

APP 1 Real-Time Database Monitoring Dashboar

Enterprise: $840–6K /yr

Replaces: Datadog DB Monitoring ($70/host/mo), SolarWinds DPA ($2K+/yr), New Relic ($50/host/mo)

A single-pane-of-glass web dashboard that monitors all your PostgreSQL, MongoDB, and MySQL/MariaDB instances in real time. It collects key metrics, stores history for trending, and fires alerts when thresholds are breached — everything Datadog or SolarWinds DPA gives you, but self-hosted and free.

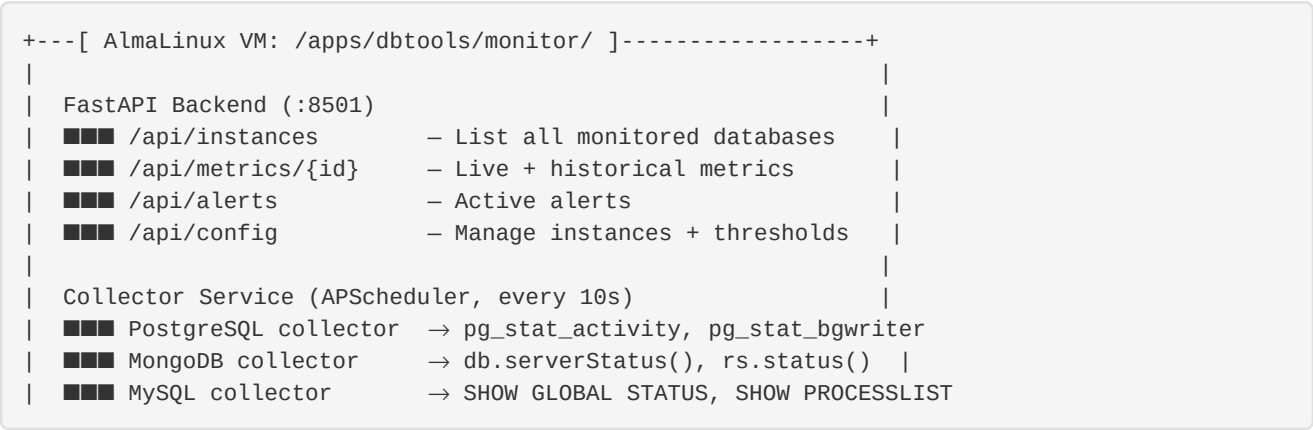
Core Features

- Multi-DB discovery — auto-detect instances from a YAML config file (host, port, credentials)
- Live metrics collection — connections, QPS, replication lag, cache hit ratio, disk usage, lock waits
- Time-series storage — SQLite or InfluxDB for metric history (7/30/90 day retention)
- Interactive charts — React + Chart.js line/area charts with zoom, auto-refresh every 10 seconds
- Alert engine — configurable thresholds with email/Slack/webhook notifications
- Instance health cards — green/yellow/red status at a glance per instance

Tech Stack

Layer	Technology	Why This Choice
Backend API	Python + FastAPI	Async support, auto-generated OpenAPI docs, lightweight
Metric Collector	Python APScheduler (background jobs)	Runs inside FastAPI, collects metrics every 10s
Database Drivers	psycopg2 (PG), pymongo (Mongo), PyMySQL (MySQL)	Standard, well-documented Python drivers
Metric Storage	SQLite (small) or InfluxDB (large)	Zero-dependency for SQLite; InfluxDB for 50+ instances
Frontend	React + Vite + Chart.js + Tailwind CSS	Fast dev, beautiful charts, responsive layout
Alerting	smtplib + requests (Slack webhook)	No external dependencies
Deployment	systemd service on AlmaLinux	Runs under /apps/dbtools/monitor/

Architecture Diagram



```
|                                                              |
|  SQLite: /apps/dbtools/monitor/data/metrics.db               |
|  React Frontend: /apps/dbtools/monitor/frontend/dist/        |
+--------------------------------------------------------------+

     Monitors:
     ■■■ PostgreSQL instances (15, 16, 17)
     ■■■ MongoDB instances (7.x)
     ■■■ MySQL / MariaDB instances (10.x, 8.x)
```

Key Metrics Collected Per Database Type

Metric	PostgreSQL	MongoDB	MySQL / MariaDB
Active connections	pg_stat_activity	serverStatus().connections	SHOW PROCESSLIST
Queries per second	pg_stat_statements	serverStatus().opcounters	SHOW GLOBAL STATUS
Replication lag	pg_stat_replication	rs.printSecondaryReplicationInfo()	SHOW SLAVE STATUS
Cache hit ratio	pg_stat_bgwriter	serverStatus().wiredTiger.cache	Innodb_buffer_pool_reads
Lock waits	pg_stat_activity (wait_event)	currentOp(waitingForLock)	Innodb_row_lock_waits
Disk usage	pg_database_size()	db.stats().storageSize	information_schema.TABLES
Slow queries	pg_stat_statements (mean_time)	system.profile	slow_query_log

Claude Code Build Prompts

Use these prompts sequentially in Claude Code to build the app iteratively:

CLAUDE CODE TIP

Prompt 1 (Scaffold): 'Create a FastAPI + React project for a database monitoring dashboard. Store it under /apps/dbtools/monitor/. Backend in Python with SQLite for metric storage, frontend with React + Vite + Chart.js. Include a config.yaml for adding DB instances.'

CLAUDE CODE TIP

Prompt 2 (Collectors): 'Add background metric collectors using APScheduler that run every 10s. Collect connections, QPS, replication lag, cache ratio, disk usage from PostgreSQL (psycopg2), MongoDB (pymongo), and MySQL (PyMySQL). Store results in SQLite metrics.db.'

CLAUDE CODE TIP

Prompt 3 (Dashboard UI): 'Build the React frontend with a grid of instance health cards (green/yello
w/red)
and detail panels with Chart.js time-series charts. Add auto-refresh every 10 seconds.
Use Tailwind CSS for styling with a dark sidebar navigation.'

CLAUDE CODE TIP

Prompt 4 (Alerts): 'Add a threshold-based alert engine. If connections > 80% of max, replication lag
 > 60s,
or disk > 85%, send alerts via email (smtplib) and Slack webhook. Store alert history in SQLite.'

APP 2 Query Performance Analyzer and Tuning A

Enterprise: $2,000–8K /yr

Replaces: SolarWinds DPA ($2K–6K/yr), EverSQL ($200–2.4K/yr), Percona PMM Enterprise ($1.2K+/yr)

A web application that captures slow queries from all your database instances, ranks them by impact, visualizes execution plans, and provides AI-powered tuning recommendations. Think SolarWinds DPA's query analysis + EverSQL's AI advisor, running locally on your infrastructure for free.

Core Features

- Slow query harvester — pulls from pg_stat_statements, MongoDB profiler, MySQL slow_query_log
- Top-N ranking — rank queries by total time, mean time, calls, rows examined, I/O
- Execution plan viewer — EXPLAIN / explain() visualization with color-coded cost nodes
- AI tuning advisor — Claude API analyzes query + schema and suggests index/rewrite improvements
- Index recommendation engine — suggests missing indexes based on query patterns
- Historical comparison — track query performance over days/weeks after changes
- Wait event analysis — categorize what queries are waiting on (I/O, lock, CPU)

Tech Stack

Layer	Technology	Purpose
Backend	Python + FastAPI	REST API for query data, explain plans, AI analysis
Query Sources	pg_stat_statements, system.profile, slow_query_log	Harvests slow queries from all 3 DB types
Plan Parser	Custom Python + JSON tree builder	Parses EXPLAIN output into renderable tree
AI Advisor	Claude API (Sonnet 4) via Anthropic SDK	Analyzes queries and recommends optimizations
Frontend	React + D3.js (plan tree) + Recharts (trends)	Interactive plan visualization and query charts
Storage	SQLite + full-text search	Query history, plans, AI recommendations

How the AI Tuning Advisor Works

```
# Backend: /api/analyze-query endpoint
import anthropic

client = anthropic.Anthropic()   # Uses ANTHROPIC_API_KEY env var

def analyze_query(query_text, explain_plan, table_schema, db_type):
    prompt = f"""You are a {db_type} performance tuning expert.

    QUERY:
    {query_text}

    EXPLAIN PLAN:
    {explain_plan}
```

```
    TABLE SCHEMA AND EXISTING INDEXES:
    {table_schema}

    Analyze this query and provide:
    1. Root cause of poor performance
    2. Specific index recommendations (CREATE INDEX statements)
    3. Query rewrite suggestions if applicable
    4. Estimated improvement factor
    """

    response = client.messages.create(
        model="claude-sonnet-4-20250514",
        max_tokens=2000,
        messages=[{"role": "user", "content": prompt}]
    )
    return response.content[0].text
```

Claude Code Build Prompts

CLAUDE CODE TIP

Prompt 1: 'Build a FastAPI app under /apps/dbtools/query-analyzer/ that connects to PostgreSQL (pg_stat_statements), MongoDB (system.profile), and MySQL (slow_query_log) to harvest slow queries. Store normalized query data in SQLite with full-text search.'

CLAUDE CODE TIP

Prompt 2: 'Add an EXPLAIN plan viewer. For PostgreSQL use EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON), for MongoDB use explain(). Parse the JSON output into a D3.js tree visualization on the React frontend with color-coded nodes (green=fast, yellow=moderate, red=slow).'

CLAUDE CODE TIP

Prompt 3: 'Integrate the Anthropic Claude API as a query tuning advisor. Send the slow query, its explain plan, and table schema to Claude Sonnet. Display the AI recommendations in a panel next to the explain plan. Cache results in SQLite to avoid duplicate API calls.'

COST SAVINGS

SolarWinds DPA: $2,000–6,000/instance/year. EverSQL: $200–2,400/year.
Your build: $0 infrastructure + ~$5–20/month Claude API for the AI advisor.
For 10 instances, this saves $20,000–60,000/year.

APP 3 Automated Backup Management and Rec

Enterprise: $3,000–15K /yr

Replaces: Commvault ($5K–15K/yr), Veeam ($3K–10K/yr), Cohesity ($5K+/yr), Rubrik ($8K+/yr)

A centralized web console that schedules, monitors, validates, and catalogs backups across all your database instances. It replaces the enterprise backup management layer while leveraging native tools (pg_dump, mongodump, mysqldump/mariadb-dump) underneath — the DBA keeps full control.

Core Features

- Unified backup catalog — every backup across all DBs logged with status, size, duration, path
- Scheduler — cron-like scheduling with full/incremental/collection-level backup policies per instance
- Automatic validation — post-backup integrity checks (restore to temp, checksum, row count)
- Retention management — auto-purge expired backups based on configurable retention policies
- One-click restore wizard — browse catalog, select backup, choose target, restore with one command
- Alerting — email/Slack on backup failure, missed schedule, or storage threshold breach
- Dashboard — backup success rate, storage consumption trends, RPO compliance status
- Restore drill tracker — log quarterly restore tests with pass/fail results

Architecture

```
+---[ /apps/dbtools/backup-console/ ]-------------------------+
|                                                             |
|  FastAPI Backend (:8502)                                    |
|  ■■■ /api/catalog         — Browse all backups (filterable)  |
|  ■■■ /api/schedule        — CRUD backup schedules            |
|  ■■■ /api/restore         — Initiate restore operations      |
|  ■■■ /api/policies        — Retention policies               |
|  ■■■ /api/dashboard       — Aggregated stats and charts      |
|                                                             |
|  Backup Executor (APScheduler + subprocess)                 |
|  ■■■ pg_dump / pg_basebackup wrapper                        |
|  ■■■ mongodump wrapper                                      |
|  ■■■ mysqldump / mariadb-dump wrapper                       |
|  ■■■ Post-backup validator (restore-to-temp + checksums)    |
|                                                             |
|  SQLite Catalog: /apps/dbtools/backup-console/catalog.db    |
|  Backups stored at: /apps/mongodb/backups/                  |
|                     /apps/pgsql/backups/                    |
|                     /apps/mysql/backups/                    |
+-------------------------------------------------------------+
```

Backup Policy Configuration Example (YAML)

```yaml
# /apps/dbtools/backup-console/policies.yaml
instances:
  - name: pg-prod-01
    type: postgresql
```

```
    host: 10.10.1.10
    port: 5432
    backup_tool: pg_dump
    schedules:
      - type: full
        cron: "0 2 * * *"         # Daily at 2 AM
        retention_days: 7
        compression: gzip
      - type: full
        cron: "0 3 * * 0"         # Weekly Sunday at 3 AM
        retention_days: 30

  - name: mongo-prod-01
    type: mongodb
    host: 10.10.1.20
    port: 27017
    backup_tool: mongodump
    schedules:
      - type: full
        cron: "0 2 * * *"
        retention_days: 7
        options: "--oplog --gzip"
      - type: collection
        cron: "0 */6 * * *"       # Every 6 hours
        database: appdb
        collection: orders
        retention_days: 3

  - name: mysql-prod-01
    type: mysql
    host: 10.10.1.30
    port: 3306
    backup_tool: mysqldump
    schedules:
      - type: full
        cron: "0 2 * * *"
        retention_days: 7
        options: "--single-transaction --routines --triggers"
```

Claude Code Build Prompts

CLAUDE CODE TIP

Prompt 1: 'Build a backup management console with FastAPI backend under /apps/dbtools/backup-console/.
Read backup policies from a YAML config. Use APScheduler to run pg_dump, mongodump, mysqldump as subprocesses on schedule. Log every backup to a SQLite catalog with status, size, duration, path.
'

CLAUDE CODE TIP

Prompt 2: 'Add a React dashboard showing: backup success/failure rate per instance (donut chart), storage consumption trend (area chart), last backup age per instance (table with color-coded staleness), and RPO compliance status (green if last backup < policy window, red if missed).'

CLAUDE CODE TIP

Prompt 3: 'Add a one-click restore wizard. User browses the backup catalog, selects a backup, chooses a target instance (same or different), and the backend runs the appropriate restore command (pg_restore, mongorestore, mysql < dump.sql) as a subprocess. Stream progress via WebSocket.'

APP 4 Database Health Audit and Compliance Ch Enterprise: $2,000–6K /yr

Replaces: ScaleGrid ($100+/mo), Idera DB PowerSuite ($2K–5K/yr), Quest Foglight ($3K+/yr)

An automated health check tool that audits every database instance against a library of best-practice rules, generates scored reports, and tracks compliance over time. It replaces tools like Idera's DB Health Check and Quest Foglight's compliance module.

Core Features

- 120+ audit rules — covering security, performance, configuration, replication, storage
- Scored reports — each instance gets a health score (0–100) with drill-down by category
- PDF report export — automated weekly reports emailed to the DBA team
- Trend tracking — health score history over weeks/months to measure improvement
- Fix-it scripts — each failed rule includes the exact command to remediate
- Multi-DB support — same rule framework for PostgreSQL, MongoDB, MySQL/MariaDB

Audit Rule Categories and Examples

Category	Example Rule (PostgreSQL)	Example Rule (MongoDB)	Example Rule (MySQL)
Security	auth enabled (pg_hba.conf has no 'trust')	authorization: enabled in mongod.conf	No anonymous users in mysql.user
Performance	shared_buffers >= 25% of RAM	WiredTiger cache sized correctly	innodb_buffer_pool_size >= 70% RAM
Indexes	No unused indexes (pg_stat_user_indexes)	No index with 0 accesses ($indexStats)	No redundant/duplicate indexes
Replication	Replication lag < 60 seconds	All RS members in PRIMARY/SECONDARY	Seconds_Behind_Master < 60
Storage	No tablespace > 85% full	No collection > 80% fragmented	No table without primary key
Config	log_checkpoints = on	operationProfiling.mode = slowOp	slow_query_log = ON
Backup	Last backup < 24 hours old	Last mongodump < 24 hours old	Last mysqldump < 24 hours old

Sample Health Report Output (JSON → PDF)

```
{
  "instance": "pg-prod-01 (PostgreSQL 16.4)",
  "scan_date": "2025-03-27T14:30:00Z",
  "overall_score": 78,
  "categories": {
    "security":    { "score": 90, "passed": 9,  "failed": 1,  "warnings": 0 },
    "performance": { "score": 65, "passed": 8,  "failed": 3,  "warnings": 2 },
    "indexes":     { "score": 70, "passed": 7,  "failed": 3,  "warnings": 0 },
    "replication": { "score": 95, "passed": 10, "failed": 0,  "warnings": 1 },
```

```
    "storage":     { "score": 80, "passed": 4,  "failed": 1,  "warnings": 0 },
    "config":      { "score": 72, "passed": 11, "failed": 3,  "warnings": 2 },
    "backup":      { "score": 85, "passed": 3,  "failed": 0,  "warnings": 1 }
  },
  "top_findings": [
    {
      "severity": "HIGH",
      "rule": "PERF-003: shared_buffers too low",
      "current": "128MB",
      "recommended": "4GB (25% of 16GB RAM)",
      "fix": "ALTER SYSTEM SET shared_buffers = '4GB'; SELECT pg_reload_conf();"
    }
  ]
}
```

Claude Code Build Prompts

CLAUDE CODE TIP

Prompt 1: 'Build a database health audit tool under /apps/dbtools/health-audit/. Define audit rules as Python classes with check(), severity, category, and fix_command properties. Start with 20 rules per database type (PostgreSQL, MongoDB, MySQL) covering security, performance, and configuration.'

CLAUDE CODE TIP

Prompt 2: 'Add a scoring engine that runs all rules against each instance, computes a weighted health score (0-100), and stores results in SQLite. Build a React dashboard with gauge charts per category, a findings table with severity badges, and instance-level trend charts over time.'

CLAUDE CODE TIP

Prompt 3: 'Add automated PDF report generation using reportlab with Carlito font. Generate a weekly health report per instance with scores, findings, and fix-it commands. Email reports via smtplib. Schedule via APScheduler to run every Monday at 8 AM.'

APP 5 Interactive Database Documentation Gene

Enterprise: $500–2K /yr

Replaces: Redgate SQL Doc ($500–1.5K/user/yr), dbForge Documenter ($250+), Dataedo ($500+/yr)

A tool that automatically introspects your database schemas — every table/collection, column/field, index, view, constraint, stored procedure — and generates beautiful, searchable, interactive documentation. It replaces expensive tools like Redgate SQL Doc while adding AI-powered descriptions.

Core Features

- Auto-introspection — connects to any instance and extracts the complete schema catalog
- AI-generated descriptions — Claude API writes human-readable descriptions for tables and columns
- Interactive web viewer — searchable, linkable, filterable schema browser with ERD-like views
- PDF/HTML export — generate downloadable documentation for offline use or audits
- Change detection — compare snapshots to detect schema drift between environments
- Data dictionary — maintain a living data dictionary with custom annotations per field
- Multi-DB support — PostgreSQL information_schema, MongoDB schema inference, MySQL metadata

Schema Extraction Sources

Information	PostgreSQL Source	MongoDB Source	MySQL Source
Databases / Schemas	pg_database, pg_namespace	listDatabases()	SHOW DATABASES
Tables / Collections	information_schema.tables	listCollections()	information_schema.TABLES
Columns / Fields	information_schema.columns	Schema inferred from sample docs	information_schema.COLUMNS
Indexes	pg_indexes	collection.getIndexes()	SHOW INDEX FROM table
Foreign Keys	information_schema.table_constraints	N/A (application-level)	information_schema.KEY_COLUMN_USAGE
Views	information_schema.views	db.getCollectionInfos({type:'view'})	information_schema.VIEWS
Stored Procedures	information_schema.routines	N/A	information_schema.ROUTINES
Row / Doc Counts	pg_class.reltuples	collection.estimatedDocumentCount()	information_schema.TABLES
Size	pg_total_relation_size()	collection.stats().storageSize	DATA_LENGTH + INDEX_LENGTH

AI-Powered Description Generation

```
# Uses Claude API to generate human-readable descriptions
def generate_descriptions(table_name, columns, sample_data):
```

```
    prompt = f"""Analyze this database table and write concise descriptions.

    Table: {table_name}
    Columns: {columns}
    Sample data (3 rows): {sample_data}

    Return a JSON object with:
    - "table_description": one-sentence description of the table's purpose
    - "columns": {{ "column_name": "description" }} for each column
    """

    response = client.messages.create(
        model="claude-sonnet-4-20250514",
        max_tokens=1500,
        messages=[{"role": "user", "content": prompt}]
    )
    return json.loads(response.content[0].text)

# Example output:
# {
#   "table_description": "Stores customer purchase orders with line items and status.",
#   "columns": {
#     "order_id": "Unique identifier for each order",
#     "customer_id": "Foreign key to the customers table",
#     "order_date": "Timestamp when the order was placed",
#     "total": "Total monetary value of the order in USD",
#     "status": "Current fulfillment status (pending/shipped/delivered/cancelled)"
#   }
# }
```

Claude Code Build Prompts

CLAUDE CODE TIP

Prompt 1: 'Build a database documentation generator under /apps/dbtools/schema-docs/. Create Python introspectors for PostgreSQL (information_schema + pg_catalog), MongoDB (listCollections + sample-based schema inference), and MySQL (information_schema). Store extracted schemas as JSON snapshots in SQLite.'

CLAUDE CODE TIP

Prompt 2: 'Build a React frontend with a left sidebar showing databases > tables/collections, and a main panel showing table details: columns with types, indexes, constraints, sample data, row counts, and size. Add full-text search across all table and column names.'

CLAUDE CODE TIP

Prompt 3: 'Add Claude API integration to auto-generate descriptions for tables and columns. Store descriptions in the SQLite data dictionary. Add a PDF export feature using reportlab that generates a complete schema reference document with all tables, columns, and descriptions.'

CLAUDE CODE TIP

Prompt 4: 'Add schema change detection. Compare the current live schema against the last stored snapshot and highlight additions, removals, and type changes. Display a diff view in the UI and optionally email a change report when drift is detected.'

Getting Started — Claude Code Workflow

Step 1: Install Claude Code

```
# Install Claude Code (requires Node.js 18+)
npm install -g @anthropic-ai/claude-code

# Or via Homebrew
brew install claude-code

# Verify installation
claude --version
```

Step 2: Set Up Your Project

```
# Create the project directory
mkdir -p /apps/dbtools/monitor     # (or whichever app you're building)
cd /apps/dbtools/monitor

# Initialize and start Claude Code
claude

# Claude Code will:
# 1. Read your working directory
# 2. Accept natural language instructions
# 3. Create files, run commands, install dependencies
# 4. Iterate based on your feedback
```

Step 3: Iterative Build Process

The recommended workflow for building each app:

- Phase 1 — Scaffold: Ask Claude Code to create the project structure, install dependencies, and wire up the basic FastAPI + React stack
- Phase 2 — Core Logic: Add the database connectors, metric collectors, or schema introspectors one database type at a time
- Phase 3 — Frontend: Build the dashboard UI with charts, tables, and forms
- Phase 4 — Polish: Add authentication, error handling, logging, and systemd service files
- Phase 5 — Deploy: Configure as a systemd service, set up log rotation, and test

Step 4: Deploy as a systemd Service

```
# Example: /etc/systemd/system/dbtools-monitor.service
[Unit]
Description=DBA Monitoring Dashboard
After=network.target
```

```
[Service]
Type=simple
User=dbtools
WorkingDirectory=/apps/dbtools/monitor
ExecStart=/apps/dbtools/monitor/venv/bin/uvicorn main:app --host 0.0.0.0 --port 8501
Restart=on-failure
RestartSec=5
Environment=ANTHROPIC_API_KEY=sk-ant-...

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable --now dbtools-monitor
sudo firewall-cmd --permanent --add-port=8501/tcp
sudo firewall-cmd --reload
```

Appendix — Project Directory Layout and Tech Summary

Recommended Directory Structure Under /apps

```
/apps/dbtools/
■■■ monitor/                    # App 1: Monitoring Dashboard
■   ■■■ main.py                 # FastAPI entry point
■   ■■■ collectors/             # PostgreSQL, MongoDB, MySQL collectors
■   ■■■ models/                 # SQLAlchemy / Pydantic models
■   ■■■ config.yaml             # Instance definitions + thresholds
■   ■■■ data/metrics.db         # SQLite time-series store
■   ■■■ frontend/               # React + Vite app
■   ■■■ venv/                   # Python virtual environment
■   ■■■ requirements.txt
■
■■■ query-analyzer/             # App 2: Query Performance Analyzer
■   ■■■ main.py
■   ■■■ harvesters/             # Slow query harvesters per DB type
■   ■■■ ai_advisor.py           # Claude API integration
■   ■■■ frontend/
■   ■■■ data/queries.db
■
■■■ backup-console/             # App 3: Backup Management Console
■   ■■■ main.py
■   ■■■ executors/              # pg_dump, mongodump, mysqldump wrappers
■   ■■■ policies.yaml           # Backup schedules and retention
■   ■■■ frontend/
■   ■■■ data/catalog.db
■
■■■ health-audit/               # App 4: Health Audit Tool
■   ■■■ main.py
■   ■■■ rules/                  # Audit rules per DB type
■   ■   ■■■ postgresql/
■   ■   ■■■ mongodb/
■   ■   ■■■ mysql/
■   ■■■ reports/                # Generated PDF reports
■   ■■■ frontend/
■   ■■■ data/audit.db
■
■■■ schema-docs/                # App 5: Documentation Generator
    ■■■ main.py
    ■■■ introspectors/          # Schema extractors per DB type
    ■■■ ai_descriptions.py      # Claude API description generator
    ■■■ snapshots/              # Schema snapshots for diff detection
    ■■■ frontend/
    ■■■ data/dictionary.db
```

Common Dependencies (Python requirements.txt)

```
# Shared across all apps
fastapi==0.115.*
uvicorn[standard]==0.32.*
pydantic==2.*
apscheduler==3.10.*
httpx==0.28.*
python-dotenv==1.0.*
pyyaml==6.*

# Database drivers
psycopg2-binary==2.9.*        # PostgreSQL
pymongo==4.10.*                # MongoDB
PyMySQL==1.1.*                 # MySQL / MariaDB

# AI integration (Apps 2 and 5)
anthropic==0.42.*              # Claude API SDK

# PDF generation (App 4)
reportlab==4.2.*

# Utilities
requests==2.32.*               # Slack/webhook alerts
```

Estimated Build Time Per App (with Claude Code)

App	Estimated Claude Code Sessions	Estimated Time	Complexity
1. Monitoring Dashboard	4–6 sessions	8–12 hours	Medium
2. Query Analyzer + AI Advisor	5–8 sessions	10–16 hours	Medium-High
3. Backup Management Console	4–6 sessions	8–14 hours	Medium
4. Health Audit Tool	5–7 sessions	10–14 hours	Medium
5. Schema Documentation Generator	4–6 sessions	8–12 hours	Medium

Total estimated effort to build all 5 apps: approximately 44–68 hours of iterative Claude Code sessions, spread across 2–4 weeks. Compared to $15,000–50,000+ per year in enterprise licenses, the ROI is immediate.

COST SAVINGS

Build all 5 apps in under 70 hours of Claude Code sessions.
Replace $15,000 - $50,000+ per year in enterprise license fees.
All tools run self-hosted on your existing AlmaLinux VMs under /apps — zero infrastructure cost.